



DIPLONAUTIC

Manual técnico

Arquitectura, stack, instalación y desarrollo

DEMO de web corporativa con área privada y foro interno

Versión del documento: 1.0

Fecha de generación: 07/10/2026

Índice

1. Introducción	3
1.1 Requisitos cubiertos	3
2. Stack tecnológico	4
2.1 Lenguajes	4
2.2 Frameworks, librerías y herramientas	4
2.3 Tamaño del proyecto	4
2.4 Justificación de las decisiones	5
3. Arquitectura	6
3.1 Flujo de una petición	6
3.2 Estructura del proyecto	7
4. Modelo de datos	7
Tabla users	8
Tabla threads	8
Tabla posts	9
5. Rutas de la aplicación	10
6. Usuarios, roles y estados	11
6.1 Decisión: alta de usuarios mixta	11
6.2 Estados de una cuenta	11
6.3 Matriz de permisos	12
7. Seguridad	12
8. Instalación y configuración	13
8.1 Variables de entorno	13
8.2 Comandos útiles	14
9. Datos y usuarios de prueba	15
10. Pruebas automatizadas	15
11. Control de versiones	16
11.1 Estrategia de ramas	16
11.2 Convención de commits	16
11.3 Pull Requests	16
11.4 Historial de commits	17
12. Uso de IA en el desarrollo	18
13. Limitaciones y mejoras futuras	18
14. Regenerar esta documentación	18

1. Introducción

Este documento describe la solución técnica de la **DEMO de web corporativa de Diplonautic**, empresa de reparación e instalación de sistemas náuticos. La DEMO incluye una parte pública de presentación de la empresa y un área privada en la que los empleados acceden con sus credenciales y participan en un **foro interno** para compartir dudas, avisos e incidencias técnicas.

El objetivo es entregar algo que funcione de principio a fin, con un código fácil de leer y explicar, un flujo de trabajo con Git ordenado y datos de prueba listos para usar.

1.1 Requisitos cubiertos

Requisito del enunciado	Implementación
Web pública: inicio y servicios	Rutas /, /servicios con contenido de ejemplo.
Página de contacto (formulario visual)	Ruta /contacto; el formulario no envía ni guarda datos.
Registro o alta por administrador	Ambos: autorregistro con correo corporativo + verificación, y alta directa desde el panel de administración.
Inicio y cierre de sesión	Sesión firmada de Flask; cierre por POST con token CSRF.
Roles empleado y administrador	Columna role con restricción CHECK y decoradores de permisos.
Usuarios verificados / no verificados	Columna is_verified; solo los verificados acceden al foro.
Foro: listado con título, autor y fecha	Ruta /foro/ con respuestas, última actividad y filtros.
Crear hilo y responder	Rutas /foro/nuevo y /foro/<id>/responder.
Solo usuarios autenticados leen y escriben	Todas las rutas del foro exigen sesión y cuenta verificada.
Datos de prueba	Usuarios en todos los estados e hilos con respuestas (app/seed.py).
Control de versiones con ramas y PR	Ramas por funcionalidad, commits descriptivos y Pull Request.

2. Stack tecnológico

Se ha elegido un stack **ligero, sin compilación y con renderizado en servidor**: arranca con un solo comando, no necesita Node.js ni un servidor de base de datos y todo el código es fácil de seguir durante una revisión en directo.

2.1 Lenguajes

Lenguaje	Dónde se usa
Python 3.14.5	Lógica de servidor, acceso a datos, pruebas y generación de manuales.
HTML5 + Jinja2	Plantillas de todas las páginas (app/templates).
CSS3	Hoja de estilos propia, responsive y sin frameworks (app/static/css/style.css).
JavaScript (vanilla)	Menú móvil y confirmación de acciones destructivas (app/static/js/main.js).
SQL (SQLite)	Esquema de la base de datos (app/schema.sql) y consultas parametrizadas.

2.2 Frameworks, librerías y herramientas

Componente	Versión	Uso
Flask	3.1.3	Microframework web: rutas, blueprints, sesiones, plantillas.
Werkzeug	3.1.9	Servidor de desarrollo y hash de contraseñas (script).
Jinja2	3.1.6	Motor de plantillas con autoescape de HTML.
Click	8.5.0	Comando de consola flask init-db.
SQLite	3.50.4	Base de datos embebida en un único fichero.
pytest	9.1.1	Pruebas automatizadas.
ReportLab	5.0.1	Generación de estos manuales en PDF.
Git	2.56.0.windows.2	Control de versiones.
GitHub	—	Repositorio remoto, Pull Requests y GitHub Actions (CI).
Google Fonts	—	Tipografías Exo 2 (títulos) e Inter (texto).

2.3 Tamaño del proyecto

Lenguaje / uso	Ficheros	Líneas
Python (backend)	8	945
Python (pruebas)	6	478
Python (documentación)	2	1080
HTML + Jinja2	15	804
CSS	1	359
JavaScript	1	17
SQL	1	48

2.4 Justificación de las decisiones

- **Flask** frente a Django: para una DEMO de este tamaño Flask permite ver todo el flujo (petición, permisos, consulta, plantilla) sin capas ocultas. Los blueprints mantienen el código separado por áreas.
- **SQLite con SQL directo** frente a un ORM: cero configuración, el esquema se lee en un único fichero y las consultas son explícitas. Todas usan parámetros (?) para evitar inyección SQL.
- **Renderizado en servidor** frente a SPA: menos piezas, funciona sin JavaScript, mejor accesibilidad y SEO para la parte pública.
- **CSS propio** frente a Bootstrap/Tailwind: la identidad visual se adapta al logotipo (azul petróleo) sin depender de un CDN ni de un proceso de compilación.
- **Alta de usuarios mixta** (ver capítulo 6): autorregistro con verificación de correo corporativo + gestión por administrador.

3. Arquitectura

La aplicación sigue el patrón **application factory** de Flask: `create_app()` crea la aplicación, carga la configuración y registra los módulos. Cada área funcional es un **blueprint** independiente:

Módulo	Responsabilidad
app/__init__.py	Fábrica de la aplicación, configuración, filtro de fechas y páginas de error.
app/public.py	Blueprint public: inicio, servicios y contacto.
app/auth.py	Blueprint auth: registro, verificación, login/logout, área privada y decoradores <code>login_required</code> , <code>verified_required</code> y <code>admin_required</code> .
app/forum.py	Blueprint forum: hilos, respuestas, búsqueda y moderación.
app/admin.py	Blueprint admin: gestión de usuarios.
app/db.py	Conexión SQLite por petición, creación del esquema y comando <code>init-db</code> .
app/security.py	Protección CSRF para todos los formularios POST.
app/seed.py	Usuarios e hilos de prueba.

3.1 Flujo de una petición

1. El navegador solicita una URL; Flask la asocia a una función de un blueprint.
2. Antes de la vista, `security.csrf_protect` valida el token en los POST y `auth.load_logged_in_user` carga el usuario de la sesión en `g.user` (si la cuenta se ha desactivado, cierra la sesión).
3. Los decoradores de permisos comprueban sesión, verificación y rol; si no se cumplen, redirigen al login, al área privada o devuelven 403.
4. La vista consulta SQLite mediante `get_db()` con consultas parametrizadas.
5. Se renderiza la plantilla Jinja2 (con autoescape) y se devuelve el HTML.

3.2 Estructura del proyecto

```
Demo Diplonautic/
|-- app/
|   |-- __init__.py      Fábrica de la aplicación
|   |-- public.py        Web pública
|   |-- auth.py          Registro, verificación, sesión y permisos
|   |-- forum.py         Foro interno
|   |-- admin.py         Gestión de usuarios
|   |-- db.py            Base de datos y comando init-db
|   |-- security.py      Protección CSRF
|   |-- seed.py          Datos de prueba
|   |-- schema.sql       Esquema SQLite
|   |-- templates/       Plantillas Jinja2 (public, auth, forum, admin)
|   |-- static/          CSS, JavaScript e imágenes
|-- tests/               Pruebas automatizadas (pytest)
|-- docs/                Manuales PDF, capturas y generador
|-- .github/             Plantilla de Pull Request y CI
|-- .devcontainer/       Entorno de GitHub Codespaces
|-- instance/            Base de datos local (no se versiona)
|-- run.py               Arranque del servidor
|-- iniciar.bat           Arranque con doble clic en Windows
|-- requirements.txt      Dependencias con versión fijada
`-- README.md
```

4. Modelo de datos

La base de datos tiene tres tablas. Un usuario puede escribir muchos hilos y muchas respuestas; un hilo tiene muchas respuestas, que se borran en cascada al eliminar el hilo. Los usuarios no se borran, se **desactivan**, para conservar la autoría de los mensajes.

users (1) ----< threads (N)	users.id = threads.author_id
users (1) ----< posts (N)	users.id = posts.author_id
threads (1) --< posts (N)	threads.id = posts.thread_id (ON DELETE CASCADE)

Tabla users

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PK autoincremental	
name	TEXT	NOT NULL	
email	TEXT	NOT NULL	Único, sin distinguir mayúsculas.
password_hash	TEXT	NOT NULL	Hash scrypt generado por Werkzeug.
role	TEXT	NOT NULL, DEFAULT 'empleado'	'empleado' o 'admin'.
department	TEXT	—	
is_verified	INTEGER	NOT NULL, DEFAULT 0	1 si confirmó el correo o lo verificó un admin.
is_active	INTEGER	NOT NULL, DEFAULT 1	0 = cuenta desactivada (no puede entrar).
verification_token	TEXT	—	Hash SHA-256 del token de verificación.
verification_sent_at	TEXT	—	Fecha de envío; el enlace caduca a las 48 h.
verified_at	TEXT	—	
created_at	TEXT	NOT NULL	
last_login_at	TEXT	—	

Tabla threads

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PK autoincremental	
title	TEXT	NOT NULL	
body	TEXT	NOT NULL	
category	TEXT	NOT NULL	'duda', 'aviso' o 'incidencia'.
author_id	INTEGER	NOT NULL, FK -> users.id	
is_pinned	INTEGER	NOT NULL, DEFAULT 0	Fijado al inicio del foro (solo admin).
is_closed	INTEGER	NOT NULL, DEFAULT 0	Cerrado: no admite respuestas.
created_at	TEXT	NOT NULL	
last_activity_at	TEXT	NOT NULL	Se actualiza con cada respuesta; ordena el listado.

Tabla posts

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PK autoincremental	
thread_id	INTEGER	NOT NULL, FK -> threads.id	Hilo al que pertenece (borrado en cascada).
author_id	INTEGER	NOT NULL, FK -> users.id	
body	TEXT	NOT NULL	
created_at	TEXT	NOT NULL	

Las tablas de este capítulo se generan automáticamente a partir de `app/schema.sql` cada vez que se ejecuta el generador de manuales.

5. Rutas de la aplicación

Tabla generada desde el mapa de URLs de Flask. **Sesión** = cualquier usuario autenticado; **Verificado** = autenticado y con el correo verificado; **Admin** = verificado y con rol administrador; **Autor o admin** = el autor del contenido o un administrador.

Método	Ruta	Acceso	Descripción
GET	/	Público	Página de inicio con presentación y servicios destacados.
GET, POST	/contacto	Público	Página de contacto. El formulario es solo visual.
GET	/servicios	Público	Catálogo de servicios y forma de trabajo.
GET, POST	/acceso	Público	Inicio de sesión.
GET	/area-privada	Sesión	Área privada: datos y estado de la cuenta.
GET, POST	/registro	Público	Registro de empleados con correo corporativo.
POST	/salir	Sesión	Cierre de sesión (solo POST).
POST	/verificacion/reenviar	Sesión	Genera un nuevo enlace de verificación.
GET	/verificar/<token>	Público	Verifica la cuenta con el token recibido por correo.
GET	/foro/	Verificado	Listado de hilos: filtro por categoría, búsqueda y paginación.
GET	/foro/<int:thread_id>	Verificado	Hilo con todas sus respuestas y formulario de respuesta.
POST	/foro/<int:thread_id>/cerrar	Autor o admin	Cierra o reabre un hilo.
POST	/foro/<int:thread_id>/eliminar	Autor o admin	Elimina un hilo y sus respuestas.
POST	/foro/<int:thread_id>/fijar	Admin	Fija o desfija un hilo.
POST	/foro/<int:thread_id>/responder	Verificado	Publica una respuesta (si el hilo no está cerrado).
GET, POST	/foro/nuevo	Verificado	Formulario y creación de un hilo.
POST	/foro/respuesta/<int:post_id>/eliminar	Autor o admin	Elimina una respuesta.
GET	/admin/usuarios	Admin	Listado de usuarios con filtros por estado.
POST	/admin/usuarios/<int:user_id>/activo	Admin	Activa o desactiva una cuenta.
POST	/admin/usuarios/<int:user_id>/enviar-verificacion	Admin	Genera un nuevo correo de verificación.
POST	/admin/usuarios/<int:user_id>/rol	Admin	Cambia el rol entre empleado y administrador.
POST	/admin/usuarios/<int:user_id>/verificar	Admin	Verifica manualmente una cuenta.
GET, POST	/admin/usuarios/nuevo	Admin	Alta directa de un usuario.

6. Usuarios, roles y estados

6.1 Decisión: alta de usuarios mixta

El enunciado permite elegir entre registro de empleados o alta por un administrador. Se han implementado **los dos**, combinados:

- **Autorregistro** restringido al dominio corporativo (@diplonautic.com, configurable con `ALLOWED_EMAIL_DOMAIN`). La cuenta nace con rol *empleado* y **sin verificar**.
- **Verificación por correo**: enlace con un token aleatorio de un solo uso que caduca en 48 h. En la base de datos solo se guarda su hash SHA-256.
- **Gestión por administrador**: puede dar de alta usuarios ya verificados, verificar cuentas pendientes, reenviar el enlace, cambiar el rol y desactivar cuentas.

Motivo: el administrador no se convierte en un cuello de botella para cada alta, pero se mantiene el control: sin correo corporativo verificado no se accede al foro, el rol de administrador solo lo concede otro administrador y cualquier cuenta se puede desactivar al momento.

Correo simulado. La DEMO no tiene servidor SMTP. El correo de verificación se muestra en pantalla (y en la consola del servidor) con el mismo contenido que se enviaría. Para producción bastaría con sustituir `render_demo_email()` por un envío real (p. ej. Flask-Mail).

6.2 Estados de una cuenta

Estado	Cómo se llega	Qué puede hacer
No verificado	Registro propio, o alta por admin sin marcar 'verificado'.	Iniciar sesión y ver su área privada. No puede leer ni escribir en el foro.
Verificado	Pulsa el enlace del correo, o un admin lo verifica.	Usar el foro completo según su rol.
Desactivado	Un administrador desactiva la cuenta.	Nada: no puede iniciar sesión y, si tenía sesión abierta, se cierra.

6.3 Matriz de permisos

Acción	Visitante	No verificado	Empleado	Admin
Ver la web pública	Sí	Sí	Sí	Sí
Registrarse	Sí	—	—	—
Entrar al área privada	No	Sí	Sí	Sí
Reenviar correo de verificación	No	Sí	—	—
Leer el foro	No	No	Sí	Sí
Crear hilos y responder	No	No	Sí	Sí
Cerrar / eliminar hilos y respuestas propios	No	No	Sí	Sí
Cerrar / eliminar contenido de otros	No	No	No	Sí
Fijar hilos	No	No	No	Sí
Gestionar usuarios	No	No	No	Sí

Un administrador no puede quitarse a sí mismo el rol ni desactivar su propia cuenta, para que siempre quede al menos un administrador operativo.

7. Seguridad

- **Contraseñas** con hash scrypt y sal (Werkzeug); nunca se guardan en claro. Política mínima: 8 caracteres combinando letras y números.
- **Mensajes de error genéricos** en el login: no revelan si un correo existe.
- **Sesión** en cookie firmada con SECRET_KEY, HttpOnly y SameSite=Lax. Se regenera al iniciar sesión (evita fijación de sesión) y el cierre de sesión es por POST.
- **CSRF**: todos los formularios POST llevan un token por sesión comparado en tiempo constante.
- **Inyección SQL**: todas las consultas usan parámetros. Los únicos fragmentos SQL dinámicos (filtros del panel) salen de un diccionario fijo, nunca de la entrada del usuario.
- **XSS**: Jinja2 escapa automáticamente todo el contenido; los mensajes del foro se muestran como texto plano.
- **Redirecciones abiertas**: el parámetro next solo acepta rutas internas.
- **Cabeceras de proxy**: X-Forwarded-* solo se tienen en cuenta con TRUST_PROXY=1 (activado en Codespaces); sin proxy delante, nadie puede falsear el dominio de los enlaces.
- **Tokens de verificación** aleatorios (secrets), de un solo uso, con caducidad y guardados con hash.
- **Control de acceso** en el servidor con decoradores, no solo ocultando botones.

Antes de un despliegue real: definir SECRET_KEY por variable de entorno, servir con HTTPS (SESSION_COOKIE_SECURE=True), usar un servidor WSGI (waitress o gunicorn) y añadir límite de intentos de login.

8. Instalación y configuración

Requisitos

- Windows, macOS o Linux con **Python 3.10 o superior** (probado con Python 3.14).
- **Git** para clonar el repositorio.
- Conexión a internet solo para instalar las dependencias y cargar las fuentes tipográficas.

Opción en la nube: GitHub Codespaces

Sin instalar nada: en la página del repositorio, pulsar **Code** → **Codespaces** → **Create codespace on main** (o el botón *Abrir en GitHub Codespaces* del README). El entorno definido en `.devcontainer/devcontainer.json` instala las dependencias, arranca la web y reenvía el puerto 5000, que se abre en el navegador. Codespaces es gratuito con un límite de horas al mes.

Opción rápida en Windows

Hacer doble clic en `iniciar.bat` en la carpeta del proyecto. El script crea el entorno virtual `.venv`, instala las dependencias, abre el navegador y arranca la web en `http://127.0.0.1:5000`.

Opción manual (cualquier sistema)

```
git clone <URL-del-repositorio> diplonautic-demo
cd diplonautic-demo
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux
pip install -r requirements.txt
python run.py                   # http://127.0.0.1:5000
```

La primera vez que arranca, la aplicación crea la base de datos `instance/diplonautic.db` con los datos de prueba. Para volver al estado inicial:

```
flask --app app init-db
```

8.1 Variables de entorno

Variable	Por defecto	Descripción
SECRET_KEY	valor de desarrollo	Clave para firmar la sesión. Obligatoria en producción.
ALLOWED_EMAIL_DOMAIN	diplonautic.com	Dominio permitido en el registro. Vacío = cualquiera.
PORT	5000	Puerto del servidor local.
HOST	127.0.0.1	Interfaz de escucha; Codespaces usa 0.0.0.0.
TRUST_PROXY	(desactivado)	Con 1 confía en las cabeceras X-Forwarded-* del proxy (Codespaces).
FLASK_DEBUG	(desactivado)	Con 1 activa el modo depuración y la recarga automática.

8.2 Comandos útiles

Comando	Para qué sirve
<code>python run.py</code>	Arranca la web en <code>http://127.0.0.1:5000</code>
<code>flask --app app init-db</code>	Borra la base de datos y la recrea con los datos de prueba.
<code>python -m pytest</code>	Ejecuta las pruebas automatizadas.
<code>python docs/generar_manuales.py</code>	Regenera los manuales PDF técnico y de usuario.

9. Datos y usuarios de prueba

Al crear la base de datos se cargan usuarios en **todos los estados** (verificado, no verificado y desactivado) y seis hilos de ejemplo con respuestas, en las tres categorías del foro, uno fijado, uno cerrado y uno escrito por la cuenta desactivada (para mostrar que su historial se conserva).

Nombre	Correo	Contraseña	Rol	Estado	Para probar
Laura Méndez	admin@diplonautic.com	Admin1234	Admin	Verificado	Panel de usuarios, moderación y fijar hilos.
Ana García	ana.garcia@diplonautic.com	Empleado1234	Empleado	Verificado	Empleada verificada: foro completo.
Carlos Ruiz	carlos.ruiz@diplonautic.com	Empleado1234	Empleado	Verificado	Segundo empleado verificado (permisos entre autores).
Marta Vidal	marta.vidal@diplonautic.com	Empleado1234	Empleado	Verificado	Empleada verificada.
Lucía Martín	lucia.martin@diplonautic.com	Empleado1234	Empleado	No verificado	Cuenta no verificada: foro bloqueado y reenvío del correo.
Javier Soler	javier.soler@diplonautic.com	Empleado1234	Empleado	No verificado	Cuenta no verificada: verificación por un administrador.
Pedro Sanz	pedro.sanz@diplonautic.com	Empleado1234	Empleado	Verificado · Desactivado	Cuenta desactivada: no puede iniciar sesión.

Credenciales ficticias, solo para la DEMO local. Los correos no existen y nunca se envía nada.

10. Pruebas automatizadas

El proyecto incluye **47 pruebas** con pytest. Cada prueba usa una base de datos temporal recién creada con los datos de prueba, por lo que son independientes entre sí.

```
python -m pytest
```

Fichero	Qué comprueba
tests/test_public.py	Páginas públicas, formulario de contacto y error 404.
tests/test_auth.py	Login y logout, cuentas desactivadas, redirección segura, registro (dominio, contraseña, duplicados), verificación (flujo completo, caducidad, un solo uso, reenvío).
tests/test_admin.py	Permisos del panel, filtros, verificar, crear, cambiar rol, desactivar y protección contra la autodesactivación.
tests/test_forum.py	Acceso solo verificado, listado con título/autor/fecha, fijados primero, crear y responder, hilos cerrados, búsqueda, escape de HTML y moderación.
tests/test_security.py	CSRF activo, cookies de sesión y contraseñas con hash.

Además, el flujo de GitHub Actions ([.github/workflows/tests.yml](https://github.com/diplonautic/diplonautic/blob/master/.github/workflows/tests.yml)) ejecuta las pruebas en cada push y en cada Pull Request.

11. Control de versiones

11.1 Estrategia de ramas

Se usa un flujo sencillo basado en ramas de funcionalidad (*feature branches*): `main` contiene siempre una versión estable y cada funcionalidad se desarrolla en su propia rama, que se integra mediante Pull Request con descripción. Las fusiones se hacen con `-no-ff` para que cada rama quede visible en el historial.

Rama	Contenido
main	Rama principal y estable.
feature/web-publica	Fábrica de la app, páginas públicas y estilos.
feature/autenticacion	Base de datos, CSRF, registro, verificación, login, roles y panel admin.
feature/foro	Foro interno: hilos, respuestas, búsqueda y moderación.
test/pruebas-automatizadas	Batería de pruebas con pytest.
docs/manuales	README, manuales PDF, plantilla de PR y CI.
feature/codespaces	Entorno de GitHub Codespaces y enlaces que funcionan tras un proxy.

Ramas presentes en el repositorio al generar este documento: `docs/manuales`, `feature/autenticacion`, `feature/codespaces`, `feature/foro`, `feature/web-publica`, `main`, `test/pruebas-automatizadas`, `origin`, `origin/docs/manuales`, `origin/feature/autenticacion`, `origin/feature/foro`, `origin/feature/web-publica`, `origin/main`, `origin/test/pruebas-automatizadas`.

11.2 Convención de commits

Mensajes en español siguiendo **Conventional Commits**: un prefijo de tipo, un ámbito opcional y un resumen en imperativo, con un cuerpo que explica el qué y el porqué.

Prefijo	Uso
feat	Nueva funcionalidad.
fix	Corrección de un error.
style	Cambios visuales o de estilos sin cambiar la lógica.
test	Pruebas automatizadas.
docs	Documentación.
ci	Integración continua.
chore	Tareas de mantenimiento (configuración, dependencias).

11.3 Pull Requests

Cada rama se integra con un Pull Request que sigue la plantilla `.github/pull_request_template.md`: resumen, cambios, cómo probarlo y checklist (pruebas en verde, sin datos sensibles, documentación actualizada). La CI de GitHub Actions ejecuta las pruebas automáticamente en cada PR.

11.4 Historial de commits

Generado con git log al construir este documento (del más reciente al más antiguo):

Commit	Fecha	Mensaje
b24ae72	2026-10-07	feat(codespaces): entorno de desarrollo para GitHub Codespaces
6ab4482	2026-10-07	fix(auth): botón de verificación con ruta relativa
f7774ec	2026-10-07	feat(app): host de escucha y confianza en proxy configurables
75a0d35	2026-10-06	Merge pull request #3 from Alecitho/docs/manuales: documentación y CI
477492e	2026-10-06	Merge pull request #2 from Alecitho/test/pruebas-automatizadas: pruebas automatizadas
015e922	2026-10-06	Merge pull request #1 from Alecitho/feature/foro: foro interno
1787e0a	2026-10-06	docs: añadir el manual técnico y el manual de usuario en PDF
c81361e	2026-10-06	docs: README con puesta en marcha, usuarios de prueba y stack
a740779	2026-10-06	docs: generador de manuales PDF con datos extraídos del proyecto
29049d0	2026-10-06	ci: ejecutar las pruebas en GitHub Actions y plantilla de Pull Request
fede495	2026-10-06	chore: script de arranque para Windows y normalización de finales de línea
01409ee	2026-10-06	fix(ui): alinear los campos de contraseña del registro
d730e1a	2026-10-06	test: pruebas automatizadas de web pública, acceso, admin y foro
68c7dde	2026-10-06	chore: ignorar la configuración local de Claude Code
24ef70a	2026-10-06	fix(ui): bordes de la tabla de usuarios y alineación de cabeceras
4202761	2026-10-06	style(foros): estilos del listado de hilos, mensajes y formularios
1c40103	2026-10-06	feat(foros): listado de hilos, creación y respuestas
1d8e86a	2026-10-06	feat(foros): tablas de hilos y respuestas con datos de prueba
f6ac88f	2026-10-06	Merge branch 'feature/autenticacion': login de empleados y roles
d5242c9	2026-10-06	style(auth): estilos de acceso, área privada y panel de usuarios
82ca125	2026-10-06	feat(admin): panel de gestión de usuarios para administradores
3477299	2026-10-06	feat(auth): registro, verificación de correo, inicio y cierre de sesión
3a079be	2026-10-06	feat(seguridad): protección CSRF en todos los formularios POST
d4782ea	2026-10-06	feat(db): esquema SQLite de usuarios y datos de prueba
ceac549	2026-10-06	Merge branch 'feature/web-publica': web pública corporativa
e5fcfba	2026-10-06	style(web): hoja de estilos corporativa y diseño responsive
8fd9778	2026-10-06	feat(web): añadir páginas públicas de inicio, servicios y contacto
49d4fcf	2026-10-06	feat(app): crear fábrica de la aplicación Flask y punto de entrada
7e49cfe	2026-10-06	chore: estructura inicial del proyecto

12. Uso de IA en el desarrollo

Siguiendo las normas de la prueba, el proyecto se ha desarrollado con apoyo de un asistente de IA (**Claude Code**, de Anthropic) integrado en el entorno de trabajo. La IA se ha usado como herramienta, con revisión humana de cada paso:

- Análisis del enunciado y propuesta de arquitectura y stack.
- Generación del código por funcionalidades, una rama cada vez.
- Revisión visual en el navegador y corrección de errores de maquetación detectados.
- Redacción de la batería de pruebas y ejecución hasta dejarla en verde.
- Redacción de los mensajes de commit, la descripción de los Pull Requests y esta documentación.

Todas las decisiones están documentadas en el código y en este manual para poder explicarlas y modificarlas en directo durante la entrevista técnica.

13. Limitaciones y mejoras futuras

Limitación actual	Mejora propuesta
El correo de verificación se muestra en pantalla.	Envío real por SMTP (Flask-Mail) o un servicio transaccional.
No hay recuperación de contraseña.	Flujo de 'he olvidado mi contraseña' con token, igual que la verificación.
Sin límite de intentos de login.	Bloqueo temporal tras varios fallos (Flask-Limiter).
Los mensajes no se pueden editar ni adjuntar ficheros.	Edición con historial y adjuntos (fotos de averías).
SQLite y servidor de desarrollo.	PostgreSQL + SQLAlchemy/Alembic y despliegue con gunicorn/waitress en Docker.
Sin notificaciones.	Avisos por correo cuando responden a un hilo propio.
Fechas con la hora local del servidor.	Guardar en UTC y mostrar en la zona horaria del usuario.

14. Regenerar esta documentación

Los dos manuales se generan con un script, por lo que pueden actualizarse tras cualquier cambio en el código:

```
python docs/generar_manuales.py
```

Salida: `docs/Manual_Tecnico_Diplonautic.pdf` y `docs/Manual_Usuario_Diplonautic.pdf`. Las rutas, el modelo de datos, las versiones, el número de pruebas, las estadísticas de código y el historial de Git se leen del proyecto en el momento de la generación.